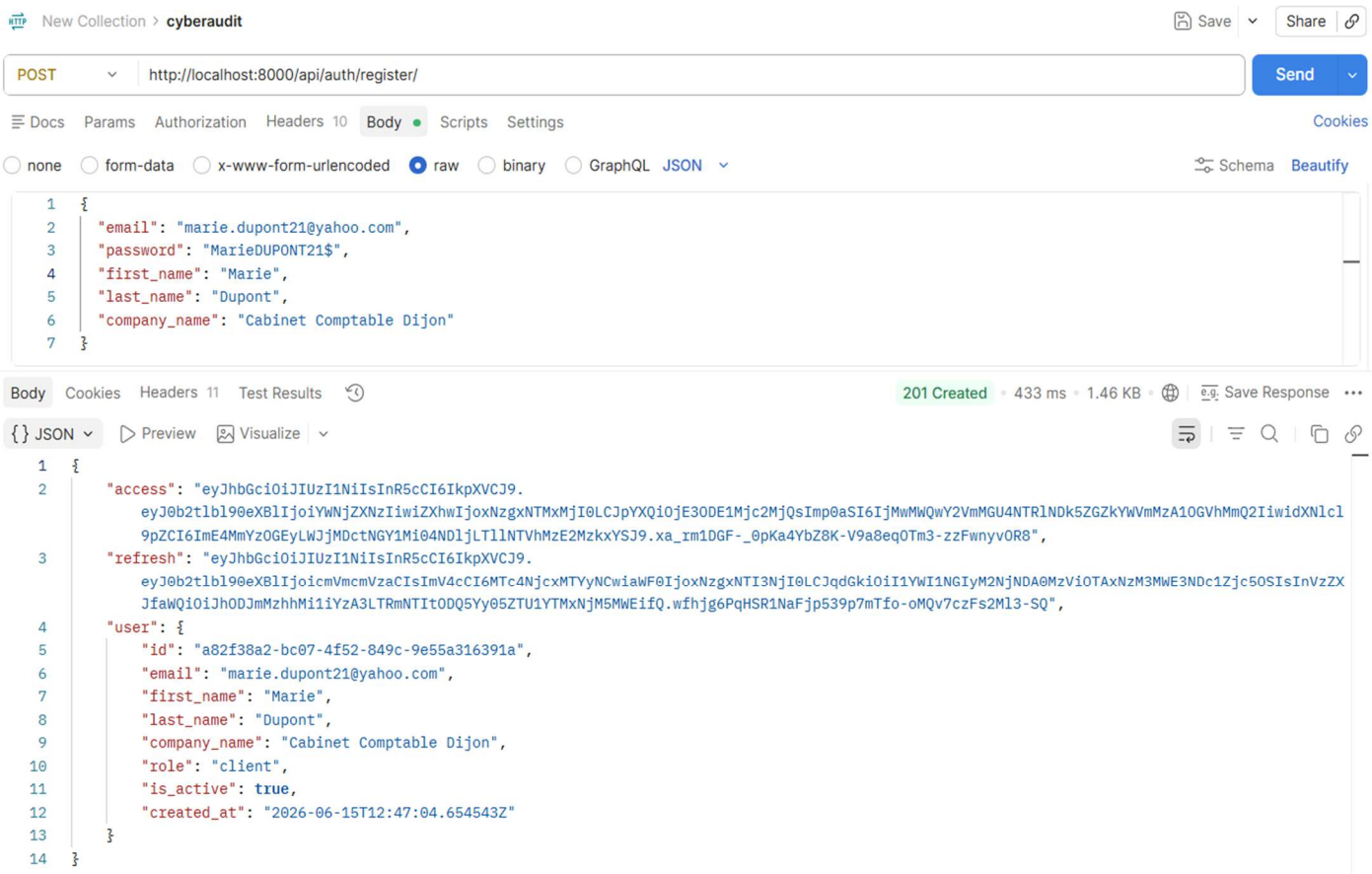
Postman test

AUTHENTICATION: /api/auth/

1) *Register (Create a customer account)*



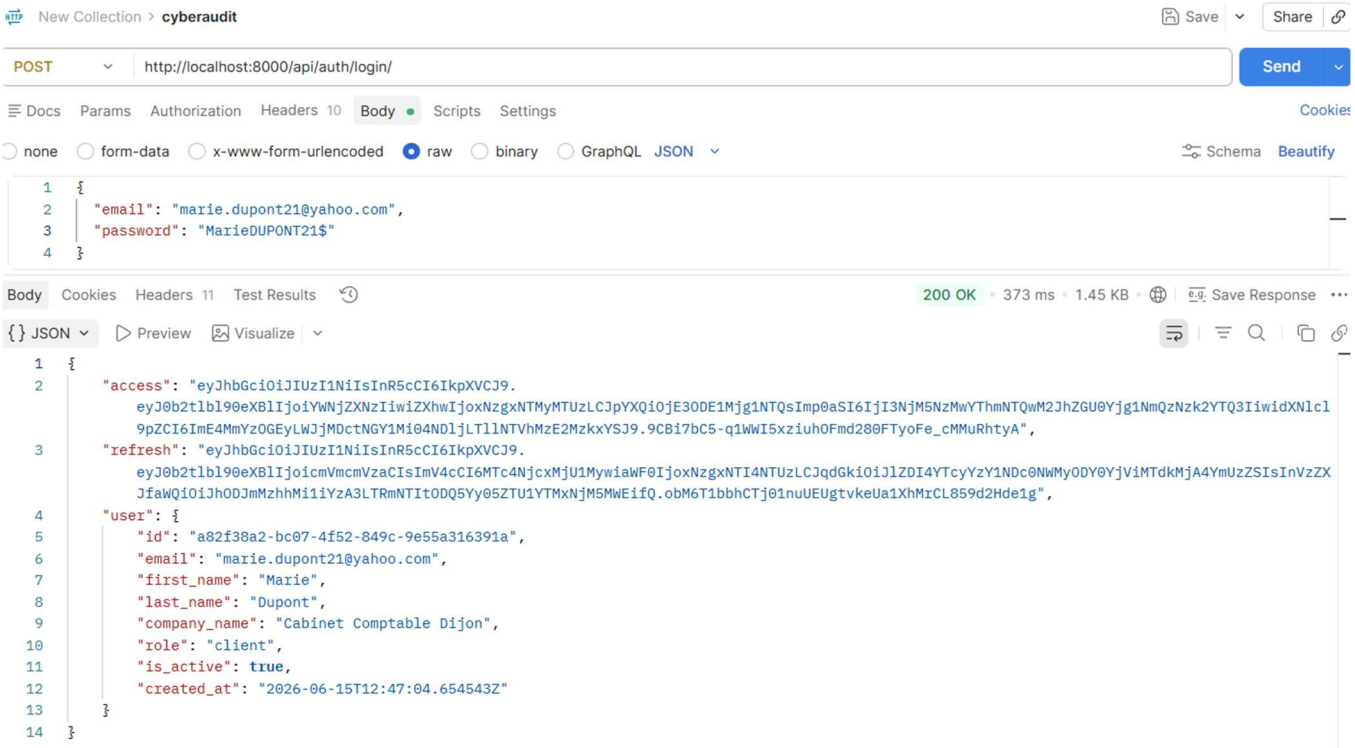
Expected result: 201 Created + {" access ": "...", " refresh ": "...", "user ": {...}}

Status 201 Created , the account has been successfully created.

What works:

- Registration creates the account with all the fields: email, first_name, last_name, company_name .
- The role is automatically assigned to "client" without having to specify it.
- is _ active : true, the account is active immediately.
- The two JWT tokens are generated directly upon registration: access and refresh.
- The ID is indeed a **UUID** : a82f38a2-bc07-4f52-849c-9e55a316391a.
- created _at is automatically registered.

2) Login



Expected result: 200 OK + JWT tokens → **copy** access **token for subsequent requests.**

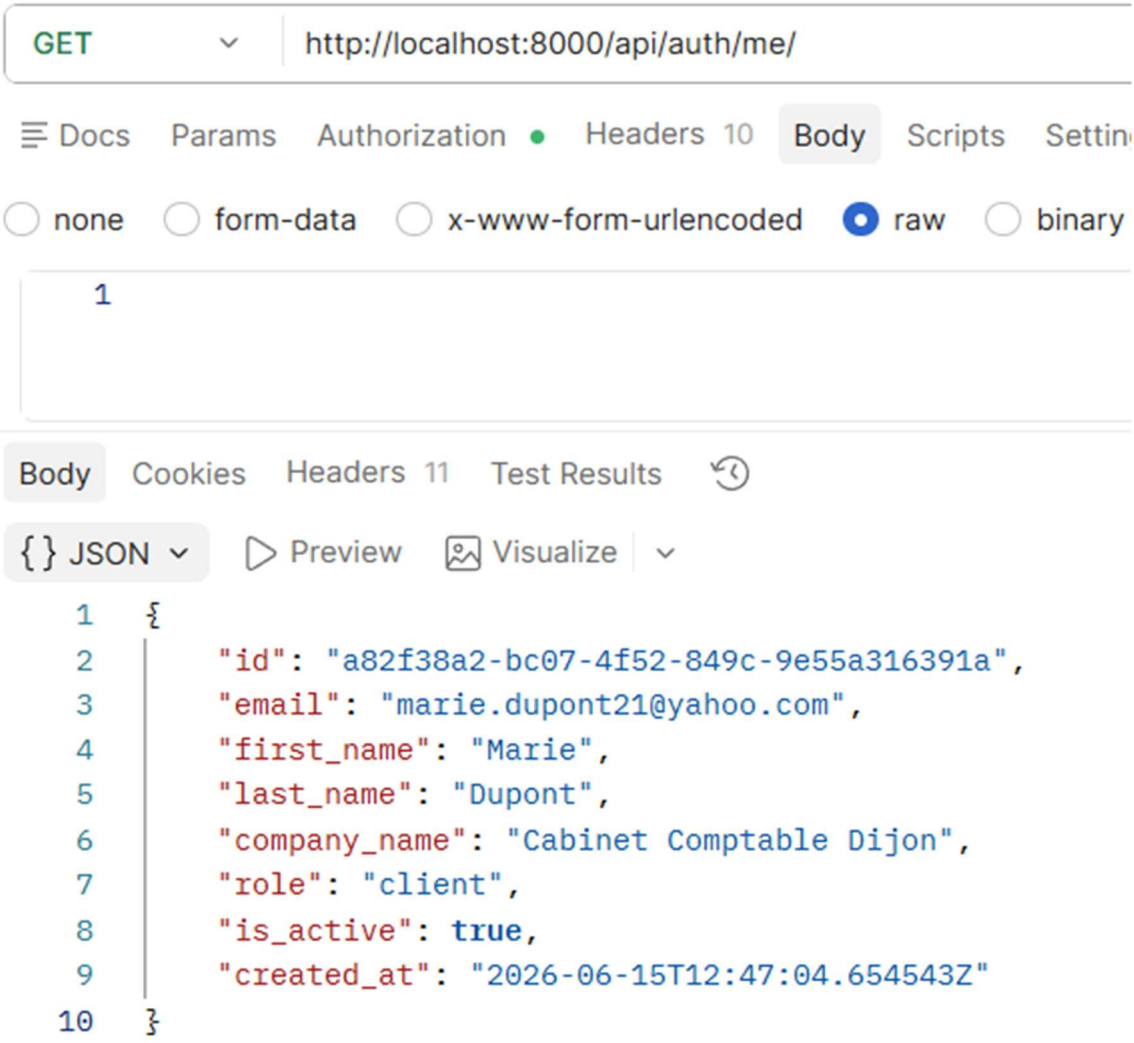
What this test shows:

- Authentication works perfectly with email and password.
- The backend returns the **two JWT tokens** at each login:
  - " access " → to be used in all protected requests (Bearer {{ token }}).
  - " refresh " → use this to renew access token when it expires.
- User data is returned directly in the response, no need to do a separate GET /me/ after login.
- The role: "client" confirms that this user does not have admin rights.

Note: the same UUID as at registration (a82f38a2-bc07-4f52-849c-9e55a316391a), it is indeed the same account.

Next step: copy the " access " token and configure it in the **Authorization** → **Bearer Token tab** of your next requests to test the protected endpoints.

3) View user



Expected result: 200 OK + logged-in user data.

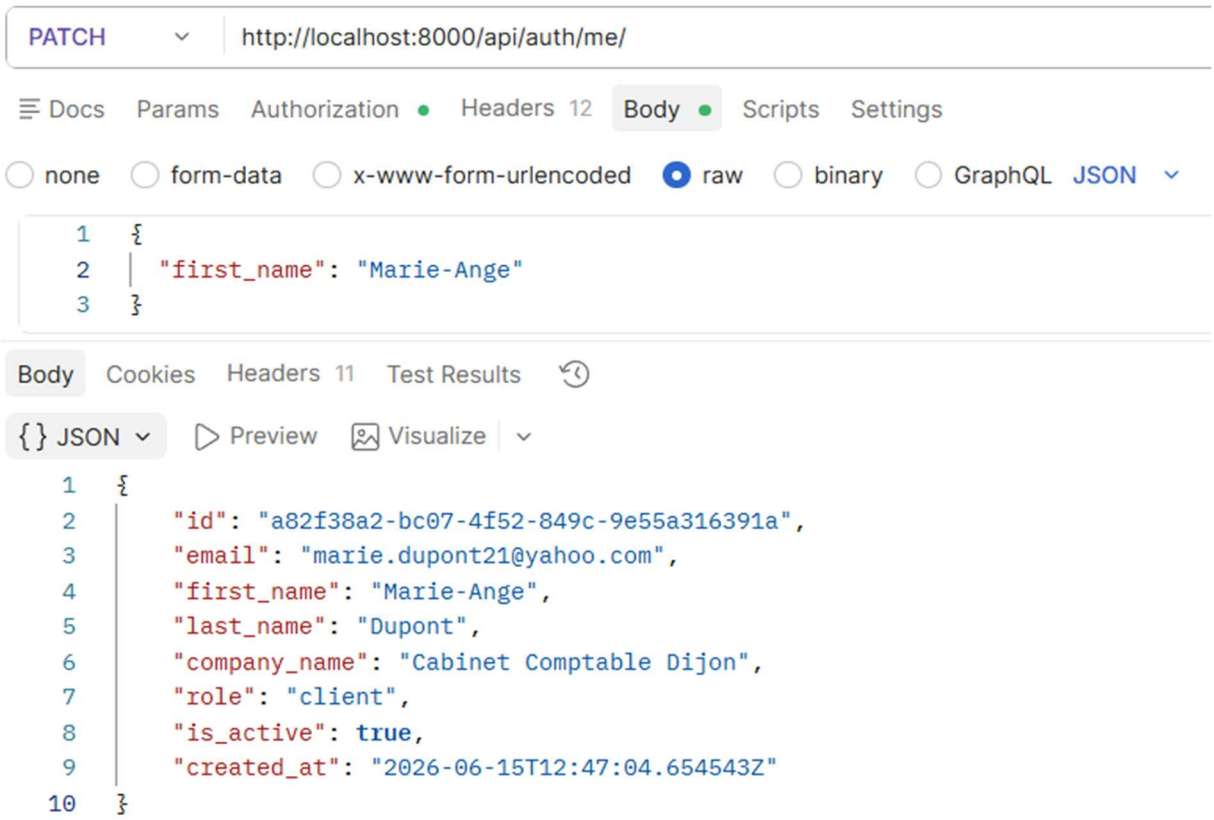
What this test shows:

- The JWT token is successfully validated; the user is identified without sending email/ password .
- All profile data is returned: id, email, first_name, last_name, company_name, role , is_active, created_at .
- The password **never appears** in the security best practice response.
- The same UUID as at registration confirms the consistency of the data.

This test therefore validates register → login → me:

The complete JWT authentication chain works end-to-end.

4) Change the user's first name



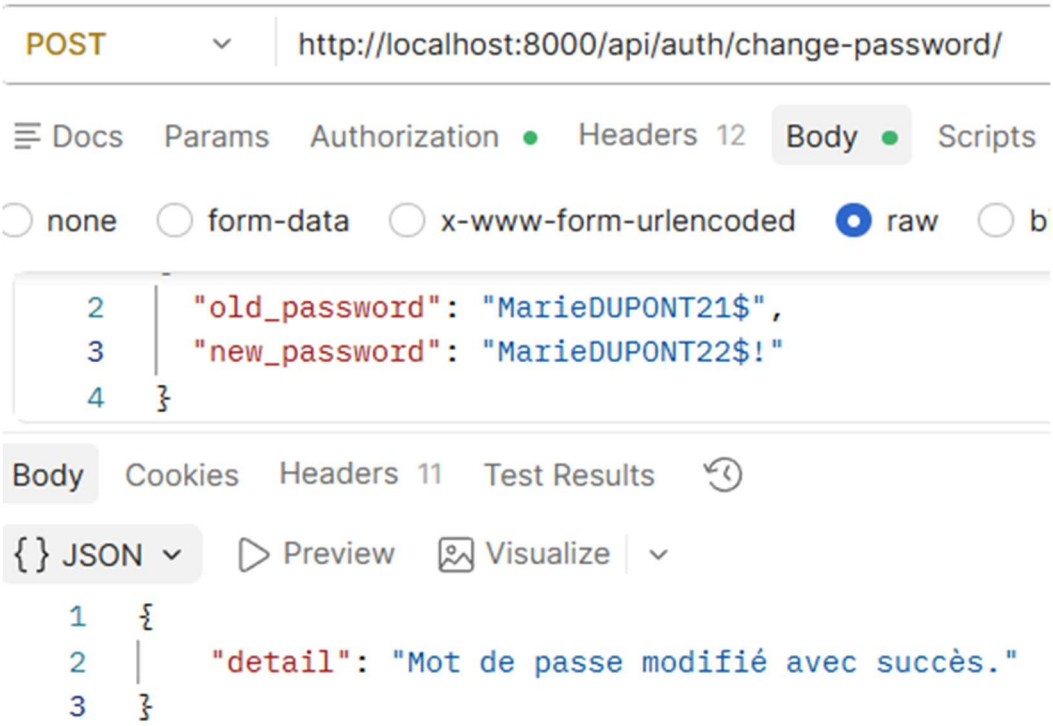
Expected result: 200 OK + profile updated.

What this test shows:

- The partial PATCH works only first_name was sent in the body, and only this field changed ("Marie" → "Marie-Ange").
- All other fields (last_name , company_name , role , etc.) are **unchanged** ; this is exactly the expected behavior of a PATCH.
- The response returns the complete updated profile immediately, without having to repeat a GET /me/.

This test is successful: the partial modification of the profile works correctly and the JWT token correctly identifies the user without passing their ID in the URL.

5) Change Password



Expected result: 200 OK + ” detail ": "Password changed successfully. " }.

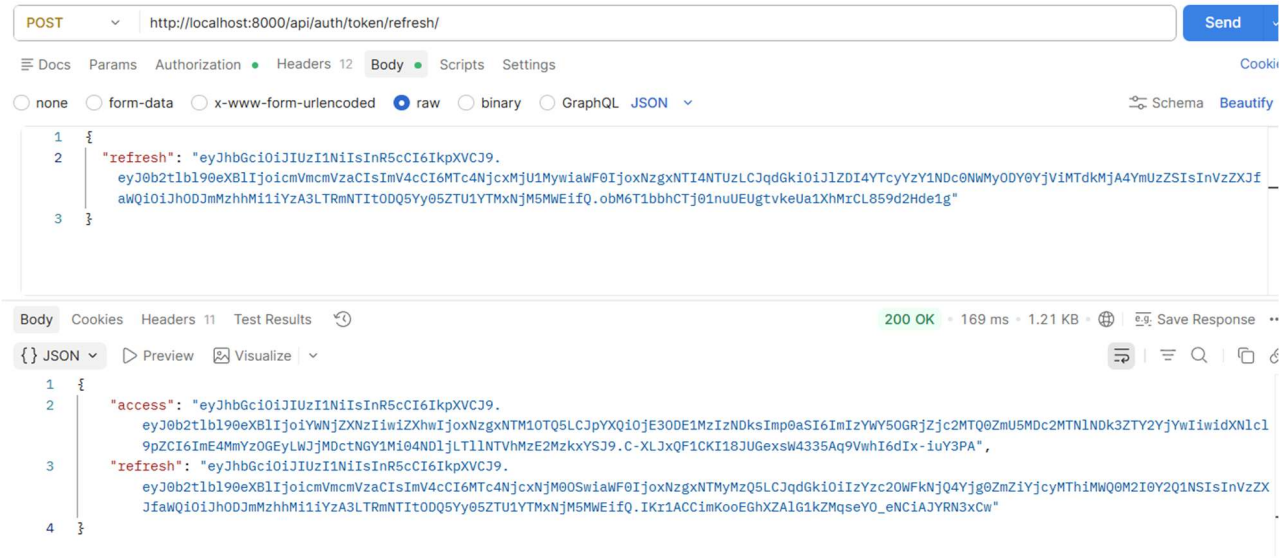
- The old password is verified before allowing the change to be made. Security is correct.

- The new password is accepted and hashed in the database.

- The response is simple and clear: "Password changed successfully." in French, consistent with the rest of the API.

- No sensitive data is returned in the response.

6) Refresh Token



Expected result: 200 OK + new access token.

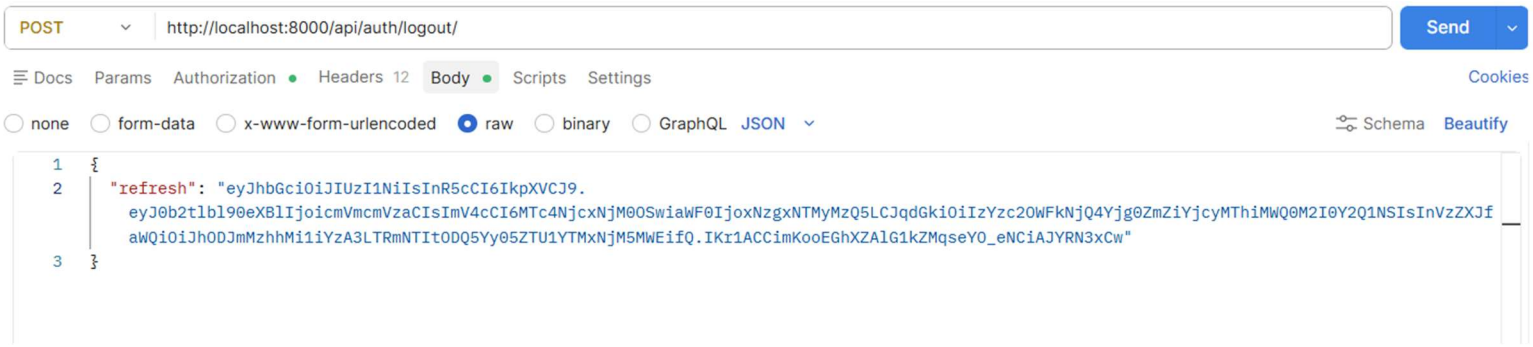
What this test shows:

- The refresh token has successfully generated a **new access** The previous **token is now replaced.**
- A **new refresh The token** is also returned; this is the ROTATE_REFRESH_TOKENS = True behavior configured in the Django settings: with each refresh , the previous refresh The token is blacklisted and a new one is issued.
- The old refresh The token sent in the body is no **longer usable** after this request.

This test is valid: automatic token renewal works correctly, useful for maintaining an active session without asking the user for their password again.

Note: Postman now replaces {{ token }} and {{ refresh }} with the new returned values.

7) Logout (Blacklist the refresh token)



Expected result: 204 No Content

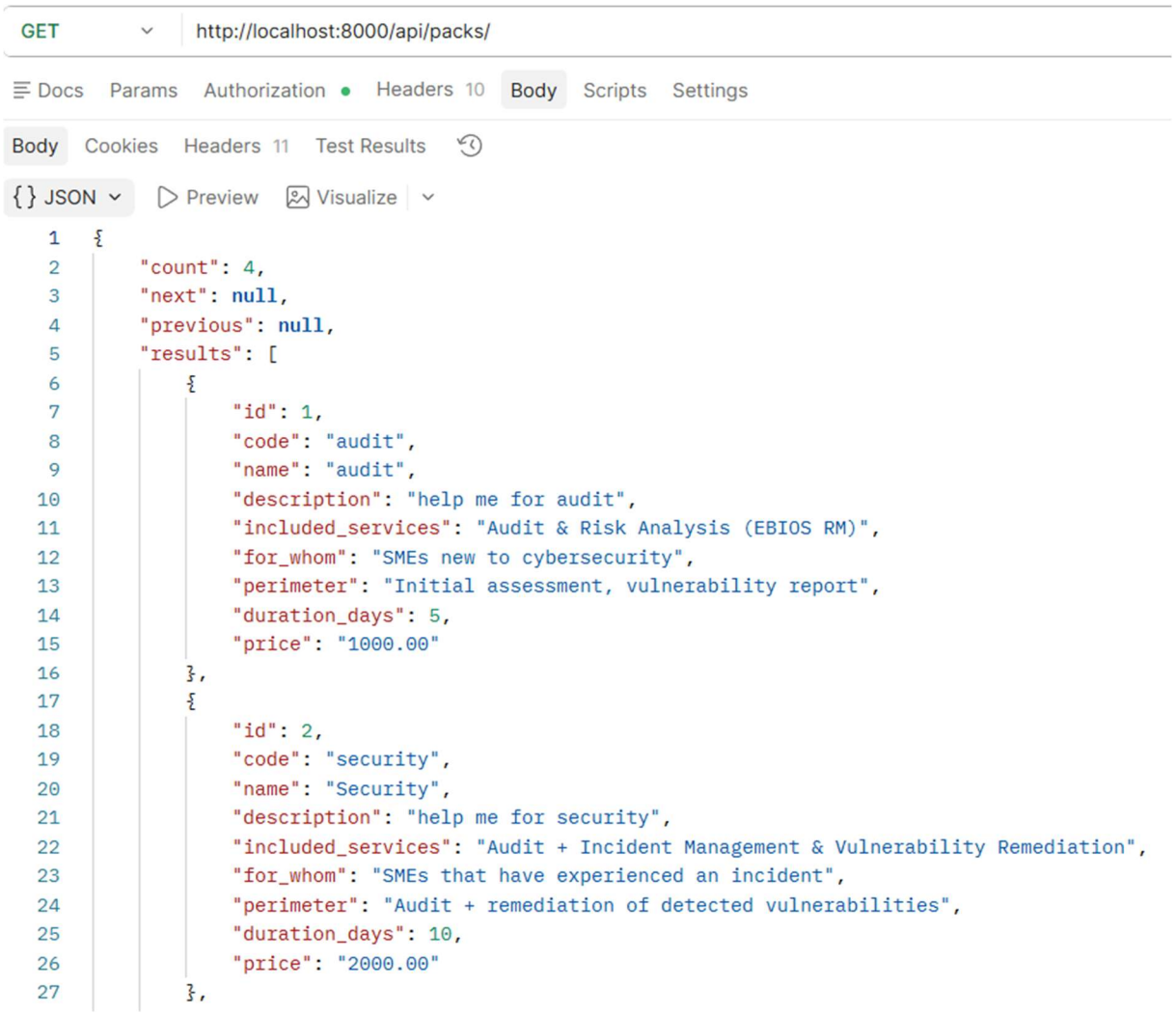
Double-check before sending:

- **Authorization** → Bearer Token with the current token.
- **Body** → the refresh token (the one obtained after the last token / refresh).

The logout should return a **204 No Content** , no body in the response, just an empty status confirming that the refresh token is blacklisted.

PACKS : /api/packs/ (public, no token required)

8) List of packs




```
{
    "count": 4,
    "next": null,
    "previous": null,
    "results": [
        {
            "id": 1,
            "code": "audit",
            "name": "audit",
            "description": "help me for audit",
            "included_services": "Audit & Risk Analysis (EBIOS RM)",
            "for_whom": "SMEs new to cybersecurity",
            "perimeter": "Initial assessment, vulnerability report",
            "duration_days": 5,
            "price": "1000.00"
        },
        {
            "id": 2,
            "code": "security",
            "name": "Security",
            "description": "help me for security",
            "included_services": "Audit + Incident Management & Vulnerability Remediation",
            "for_whom": "SMEs that have experienced an incident",
            "perimeter": "Audit + remediation of detected vulnerabilities",
            "duration_days": 10,
            "price": "2000.00"
        },
```

```
        {
            "id": 3,
            "code": "protection",
            "name": "Protection",
            "description": "help me for protection",
            "included_services": "Audit + System Security & 24/7 SOC Monitoring",
            "for_whom": "SMEs with sensitive data",
            "perimeter": "Audit + Continuous Monitoring & Real-Time Alerts",
            "duration_days": 15,
            "price": "3500.00"
        },
        {
            "id": 4,
            "code": "premium",
            "name": "Prenium",
            "description": "help me for prenium",
            "included_services": "Audit + Incidents + SOC + Team Awareness",
            "for_whom": "SMEs seeking comprehensive coverage",
            "perimeter": "The most comprehensive all-in-one solution",
            "duration_days": 20,
            "price": "5000.00"
        }
    ]
}
```

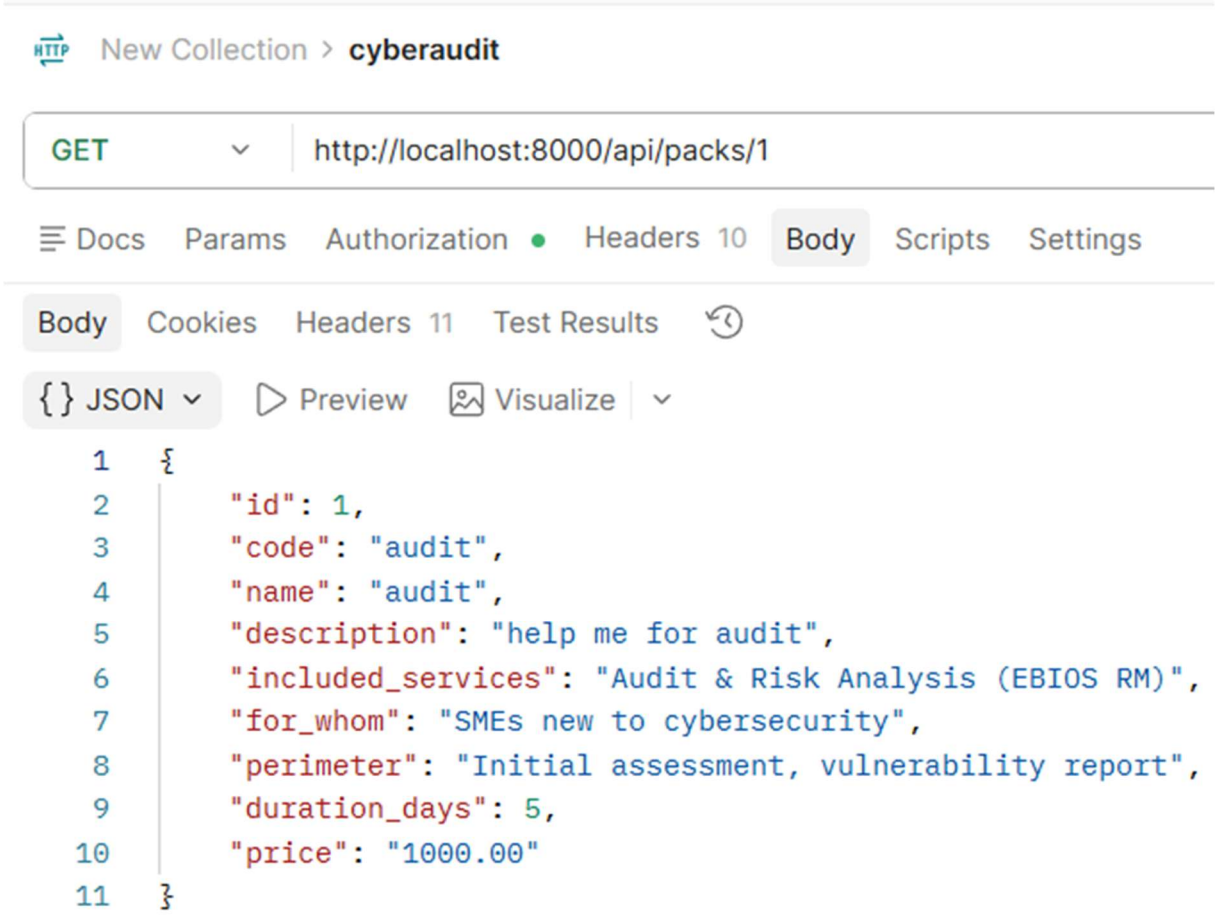

Expected result: 200 OK + list of 4 packs (Audit, Security, Protection, Prenium).

- Pagination is working (next : null , previous : null (everything fits on one page).
- Each pack contains all the expected fields: id, code, name , description, included_services , for_whom , perimeter , duration_days , price .
- This endpoint is **public** , no token required, anyone can see the offers.

The 4 packs created:

id	code	price	duration
1	audit	€1,000	5 days
2	security	€2,000	10 days
3	protection	€3,500	15 days
4	premium	€5,000	20 days

9) Pack detail



Expected result: 200 OK + package details (name , included_services, price , duration_days…).

What this test shows:

- The endpoint returns the full details of a single pack by its id.
- **Public** endpoint , no token required.
- All the information is there: included services, target (for_whom), scope, duration, price.

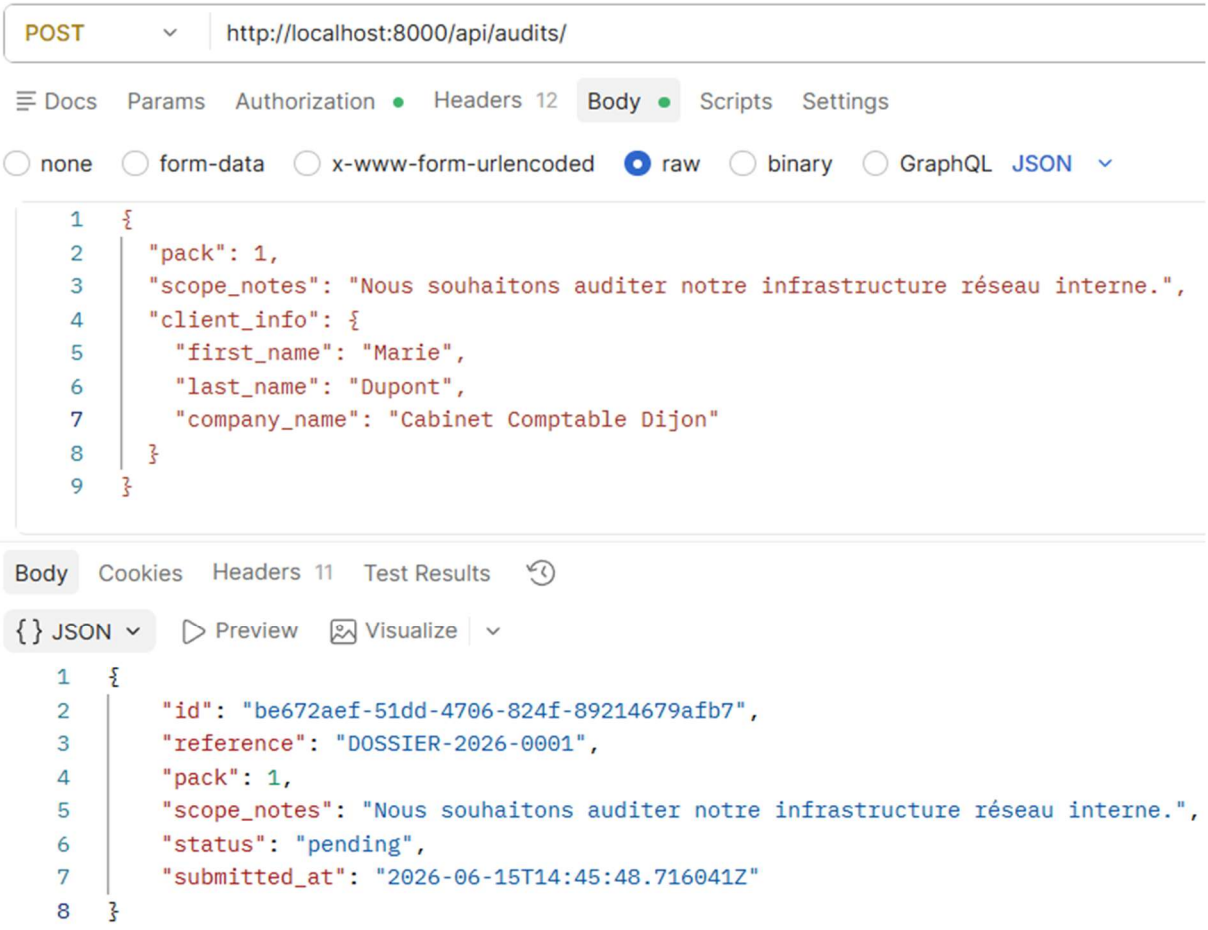
Note: The URL /api/packs/1 without a trailing slash works here because it's a GET request. Django can redirect automatically. But as a best practice, always use the slash: http://localhost:8000/api/packs/1/

AUDIT REQUESTS: /api/audits/

10) Create an audit request (client only)

http://localhost:8000/api/audits/

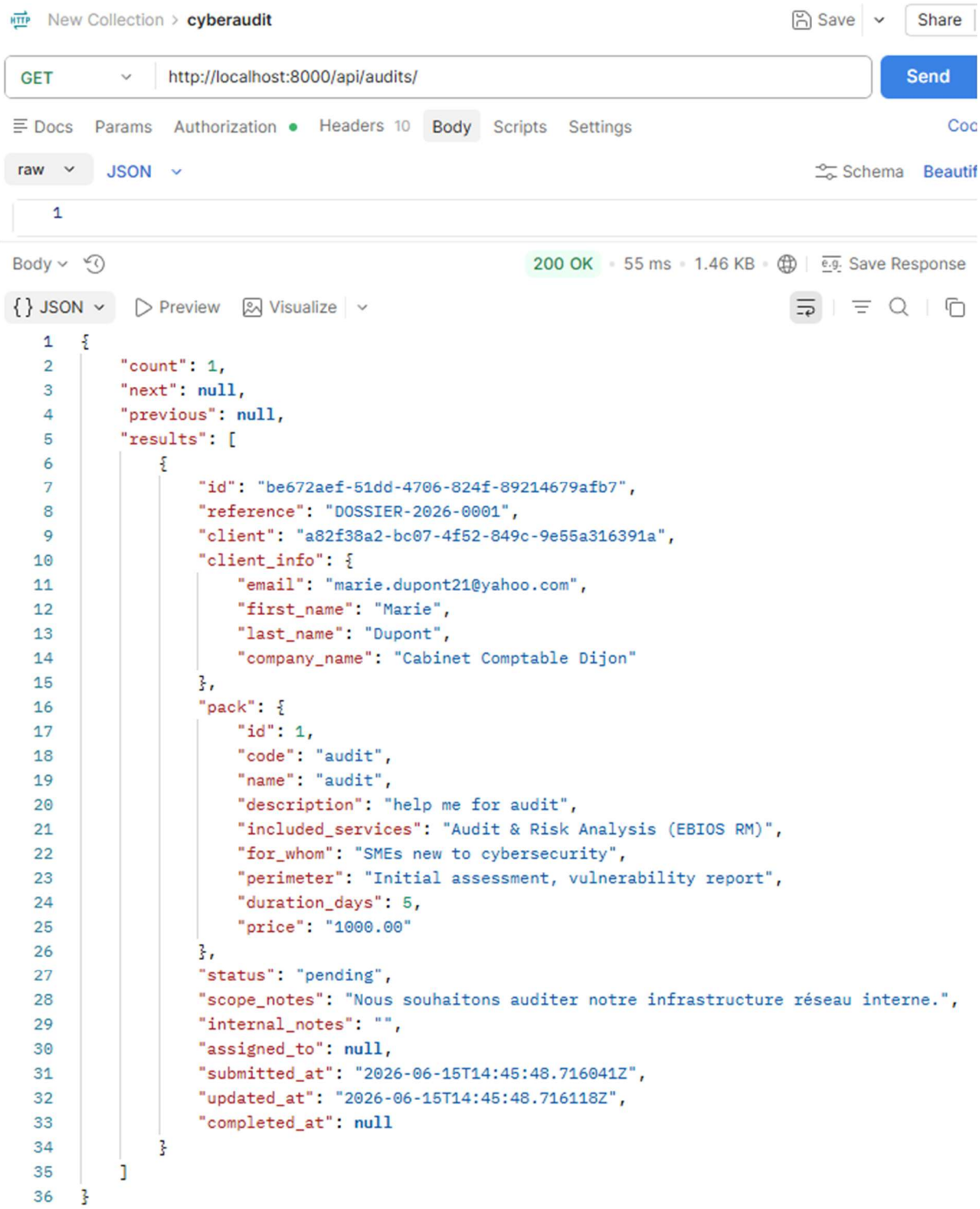
GET http://localhost:8000/api/packs/



- The application is correctly registered with a **unique UUID** : be672aef-51dd-4706-824f-89214679afb7.
- A **readable reference** is automatically generated: "DOSSIER-2026-0001" convenient for communicating with the client.
- The initial status is " pending ", awaiting processing by the admin.
- submitted_at is automatically timestamped.
- The JWT token allowed Marie to be automatically identified as a client; there was no need to send her ID in the body.

Note: The client_info fields sent in the body (first_name, last_name, company_name) are not returned in the response; they are likely ignored by the serializer because the client is identified via the token. Only pack, scope_notes, and the automatically generated fields are returned.

11) List of audit requests

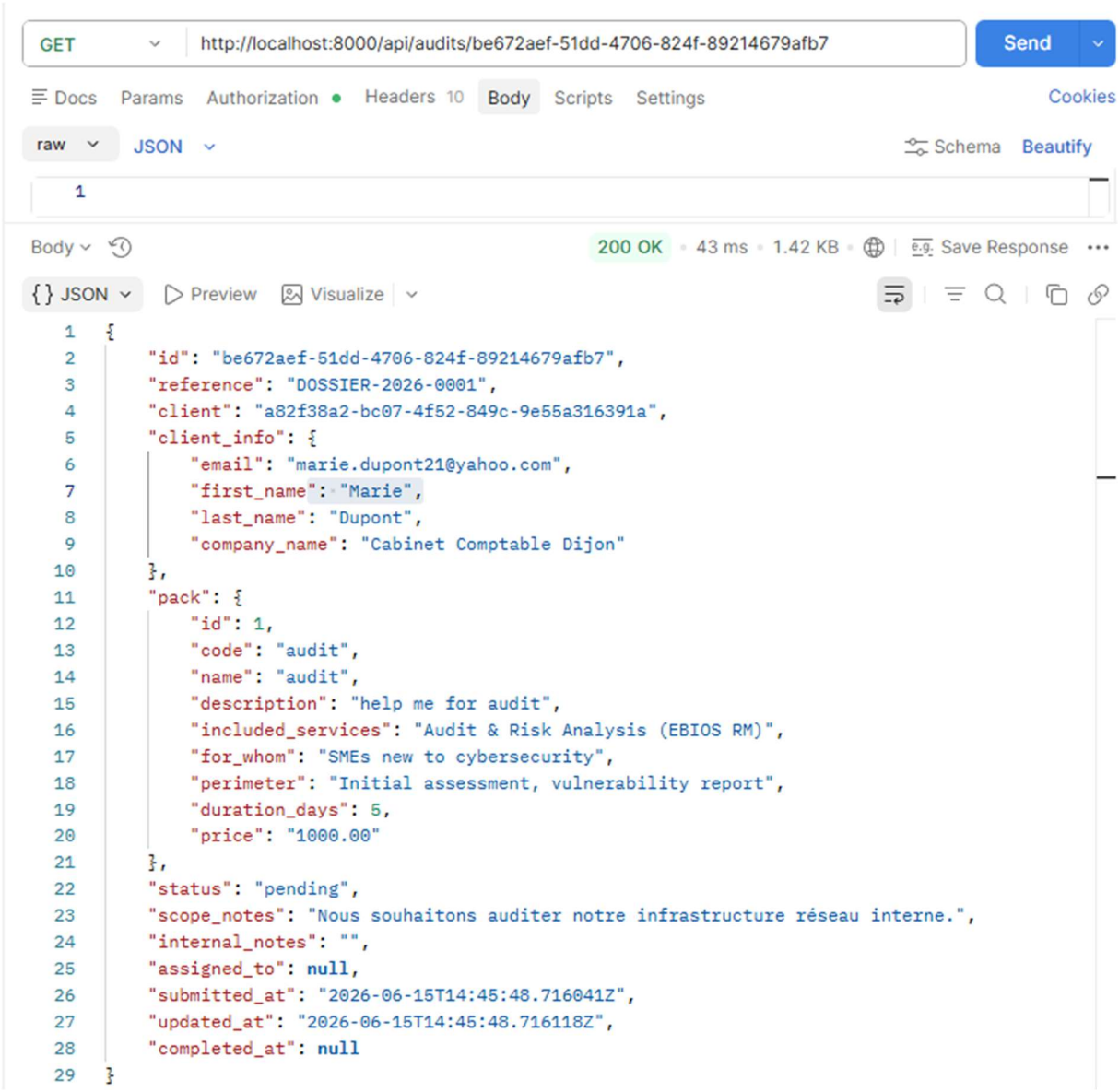


- "count ": 1) Marie **only sees her own requests** (RBAC client), not those of other clients.
- The response is **enriched** compared to the creation: the pack is now a complete object with all its details, and client_info displays the client's data.
- tracking fields are visible : status, internal_notes , assigned_to , submitted_at , updated_at , completed_at

Important fields for the rest of the tests:

Field	Value
id	be 672aef-51dd-4706-824f-89214679afb7
reference	FILE-2026-0001
status	pending
assigned _to	null → not yet assigned to an admin
internal _notes	"" → empty, reserved for admin
completed _at	null → not finished yet

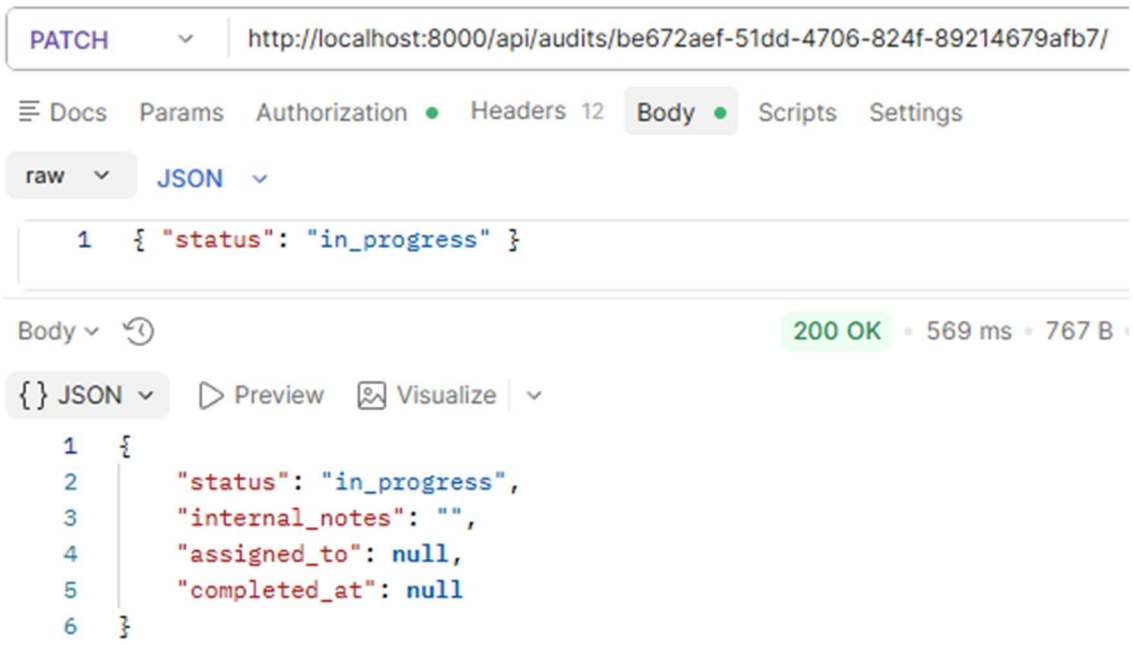
12) Details of a request



- The UUID in the URL correctly returns **a single request** with all its complete data.
- Same result as the list but without the pagination directly to the object.
- Marie can access her own request via her client token .

Difference with the previous test (list): here we access **a specific request** by its UUID, useful when the frontend wants to display the details page of a folder.

13) Change status (admin only)



What this test shows:

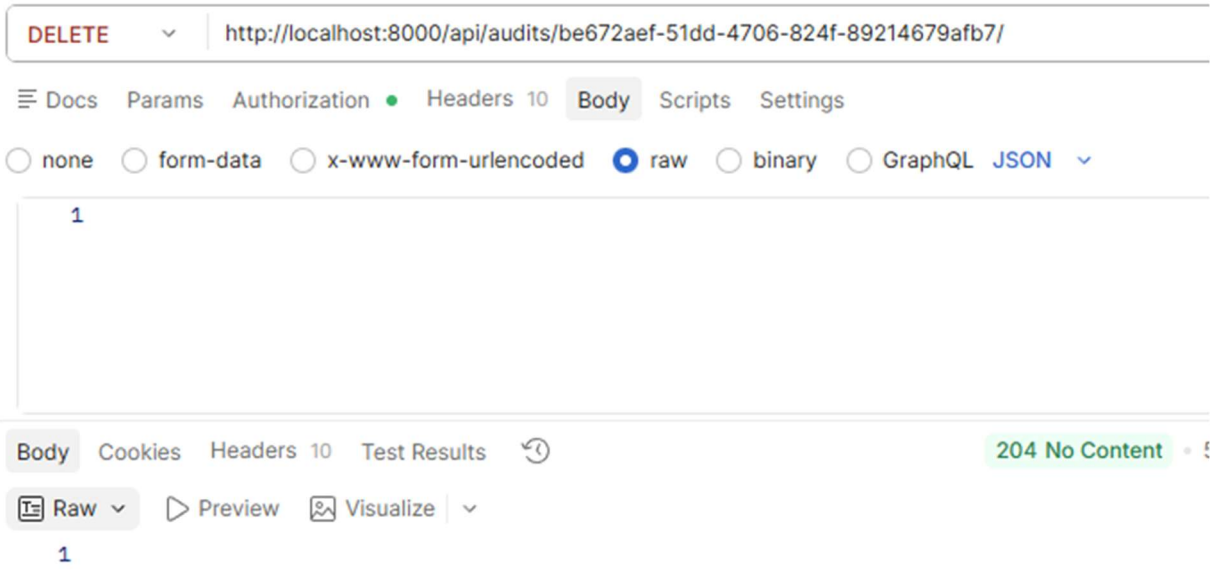
- The status has successfully changed from " pending " → " in_progress ".
- Only fields **that can be modified by the admin** are returned: status , internal_notes , assigned_to , completed_at , this is the AuditRequestAdminUpdateSerializer in action.
- The RBAC works: only an admin token can make this PATCH (a client token would return 403).

What happened in the background: the backend automatically sent an **email notification to Marie** to inform her of the status change; this is the perform_update and create_and_send logic in views.py.

14) Archive a request (admin only)

DELETE http://localhost:8000/api/audits/{id}/

Auth: Bearer {{ token_admin }}



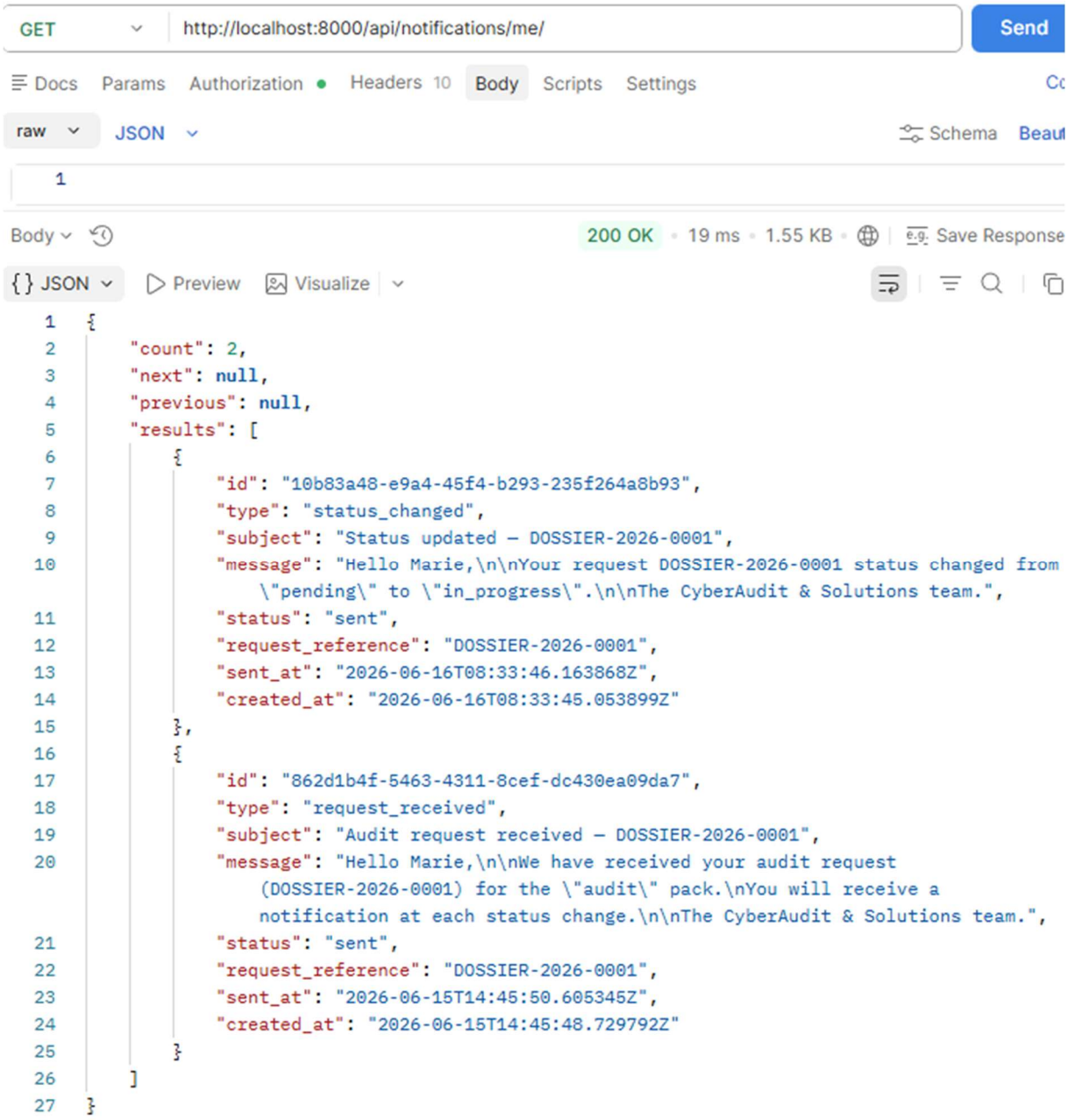
- Status 204 means success **without a body in response;** this is the standard behavior for a DELETE.
- The request is **not deleted** from the database, it is **archived** (soft delete): the status changes to " archived " in the background.
- Only one token **The admin** can perform this action; the RBAC is working correctly.

Why a soft delete ? It's an important architectural choice: audit data is never truly deleted for reasons of traceability and legal compliance. The administrator "archives" a folder rather than permanently deleting it.

15) User notifications

GET http://localhost:8000/api/notifications/me/

Auth: Bearer {{token}}



What this test shows:

Marie received 2 automatic notifications related to her file DOSSIER-2026-0001:

Notification 1 “ status_changed (today)”:

- Sent when the admin changed the status from pending to in_progress .
- Message : *"Your request DOSSIER-2026-0001 status changed from 'pending' to ' in_progress'".*

Notification 2 “ request_received (yesterday)”:

- Sent automatically when the request is created.
- Message: *"We have received your audit request for the 'audit' pack".*

What this validates:

- The notification system works end-to-end; every action automatically triggers a notification.
- Notifications are linked to the user via the token ; Marie only sees her own.
- The status field : "sent" confirms that the emails have been sent.
- The timestamps sent_at and created_at precisely track each event.

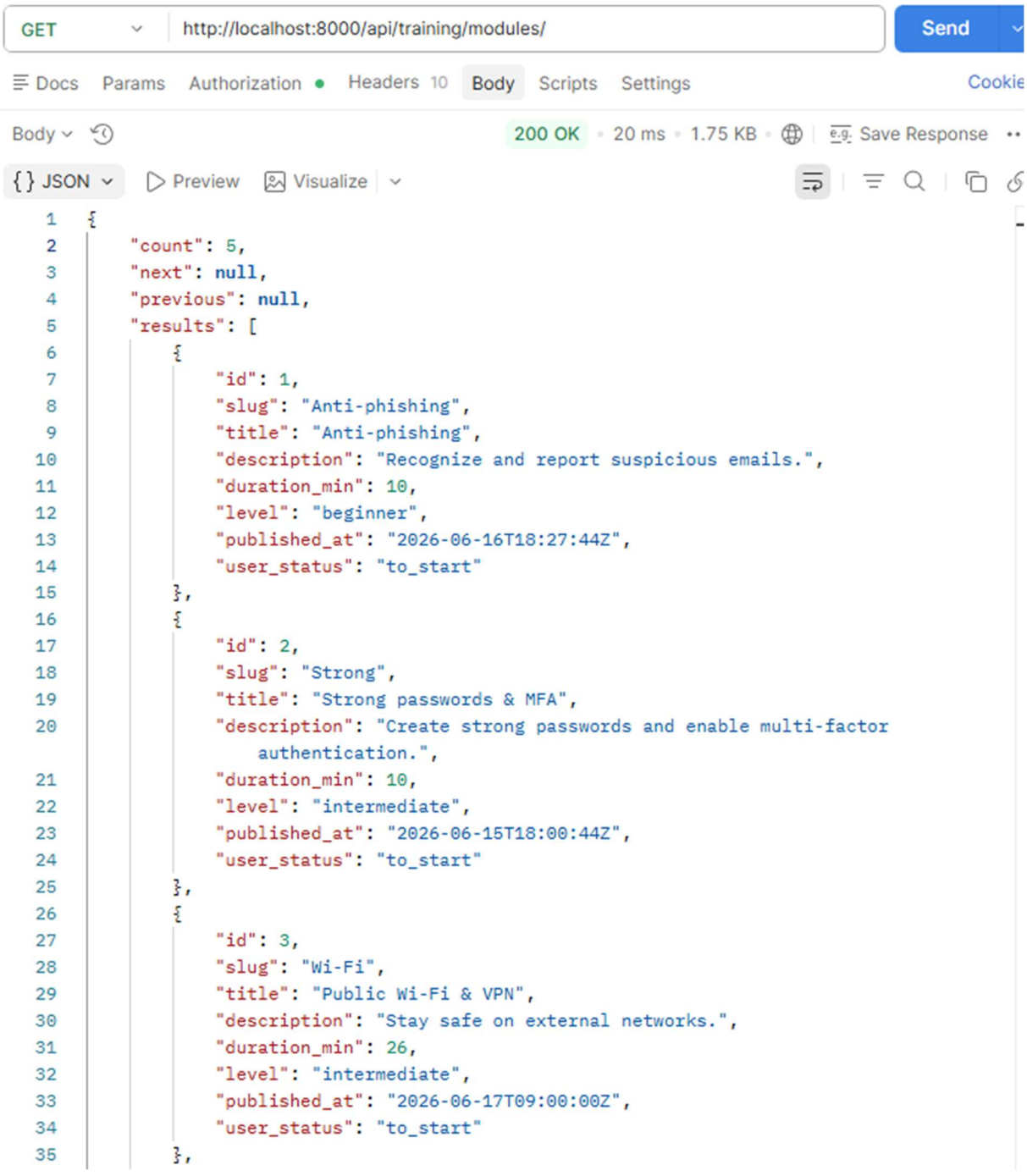
This is one of the most important features of the application: complete traceability of the lifecycle of an audit file.

Training

16) List of modules

GET http://localhost:8000/api/training/modules/

Auth: Bearer {{ token }}



```
        {
            "id": 4,
            "slug": "Data",
            "title": "Data backup",
            "description": "Best practices for backing up business data.",
            "duration_min": 10,
            "level": "advanced",
            "published_at": "2026-06-17T12:00:00Z",
            "user_status": "to_start"
        },
        {
            "id": 5,
            "slug": "Incident",
            "title": "Incident response",
            "description": "What to do when something goes wrong.",
            "duration_min": 10,
            "level": "beginner",
            "published_at": "2026-06-20T10:34:24Z",
            "user_status": "to_start"
        }
    ]
}
```


All 5 complete modules are visible:

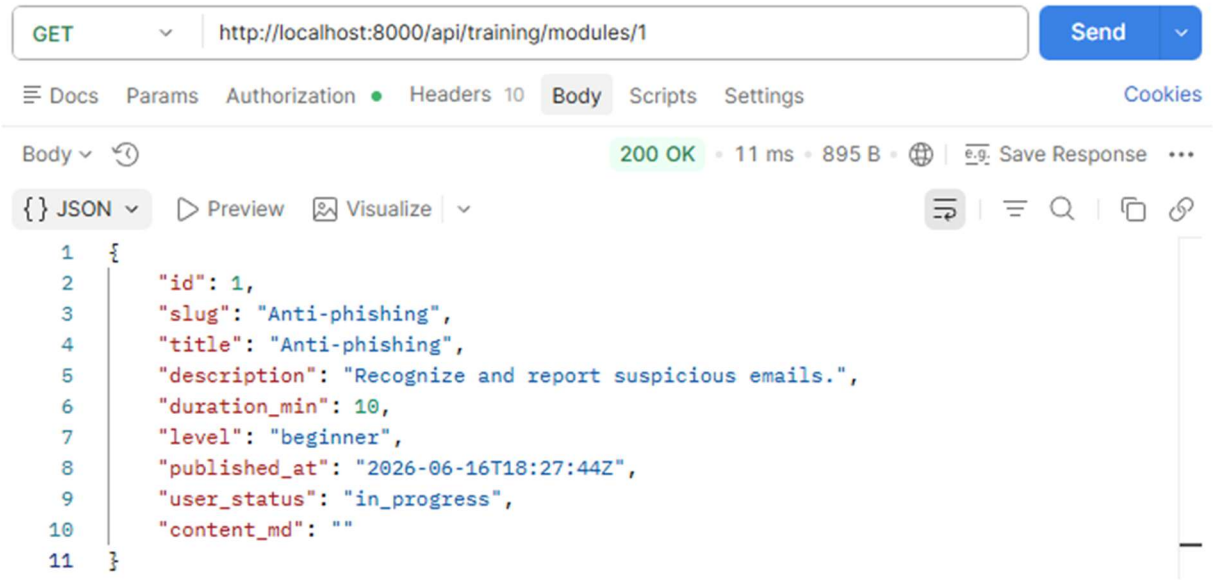
id	title	level	duration
1	Anti-phishing	beginner	10 min
2	Strong passwords & MFA	intermediate	10 min
3	Public Wi-Fi & VPN	intermediate	26 min
4	Data backup	advanced	10 min
5	Incident response	beginner	10 min

Which is perfect: all user_status are " to_start " for Marie, consistent because she has not yet started any module.

17) Detail of a module

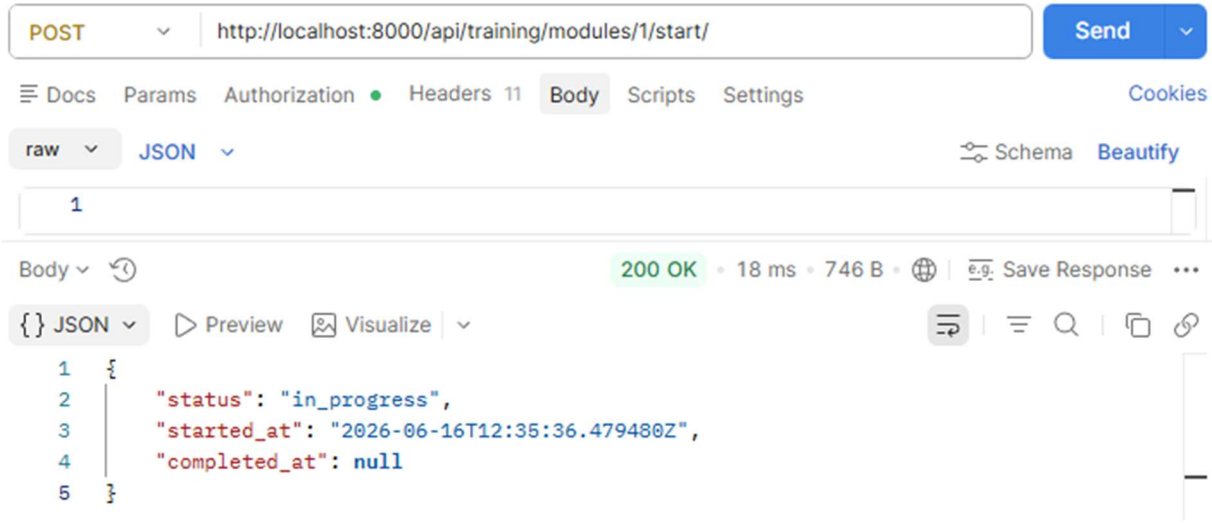
GET http://localhost:8000/api/training/modules/1/

Auth: Bearer {{ token }}



- The user _status is now " in_progress ", Marie has successfully started the Anti-phishing module.
- The field " content_md ": "" appears only in the detail view (not in the list), it is the content of the module in Markdown , empty for now because it has not been entered in the admin.
- Progress is **persisted in the database** ; if Marie closes the app and comes back, her in_progress status will still be there.

18) Start a module

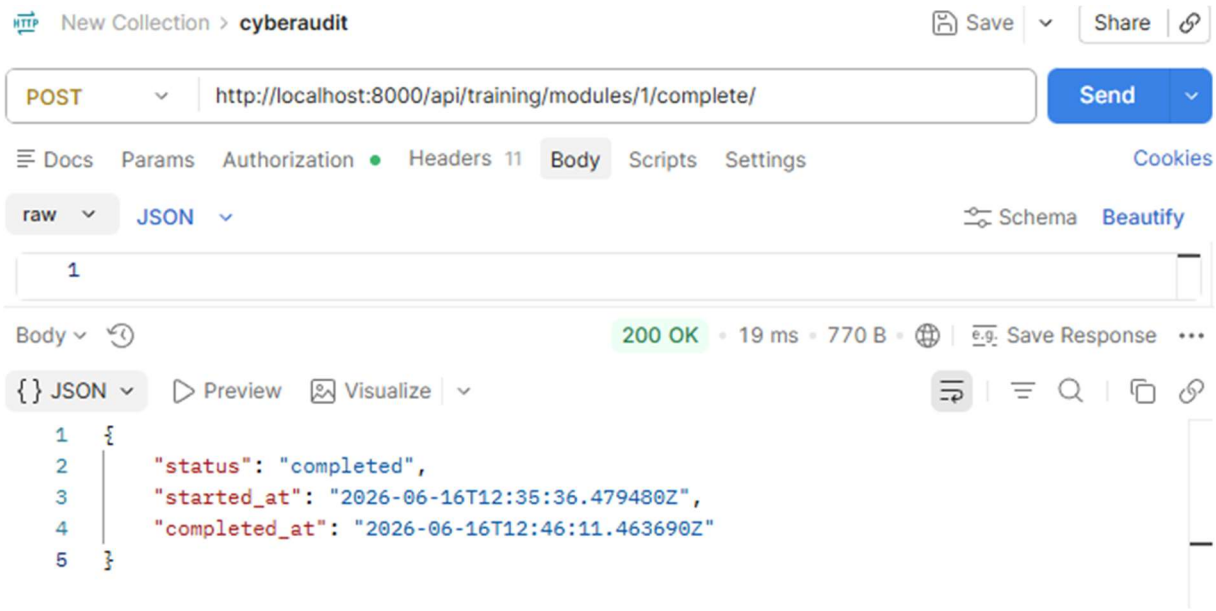


- The progress status changes to " in_progress ".

- started_at is automatically timestamped: 2026-06-16T 12:35:36 .

- completed_ at : null , the module is not yet finished.

19) Complete a module

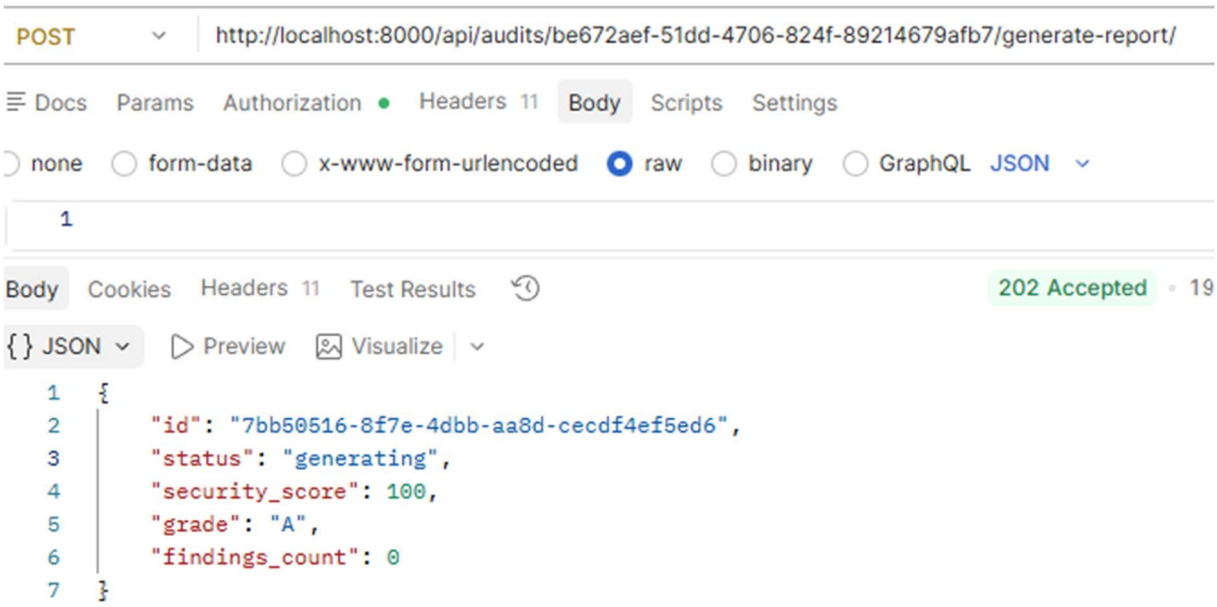


What this test shows:

- The status has changed to " completed ".
- started _at : 12:35: 36 : start time.
- completed _at : 12:46: 11 : end time.
- Marie took **10 minutes 35 seconds** to complete the Anti-phishing module, consistent with duration_ min : 10.

The complete lifecycle of a module is validated: to_start → in_progress → completed .

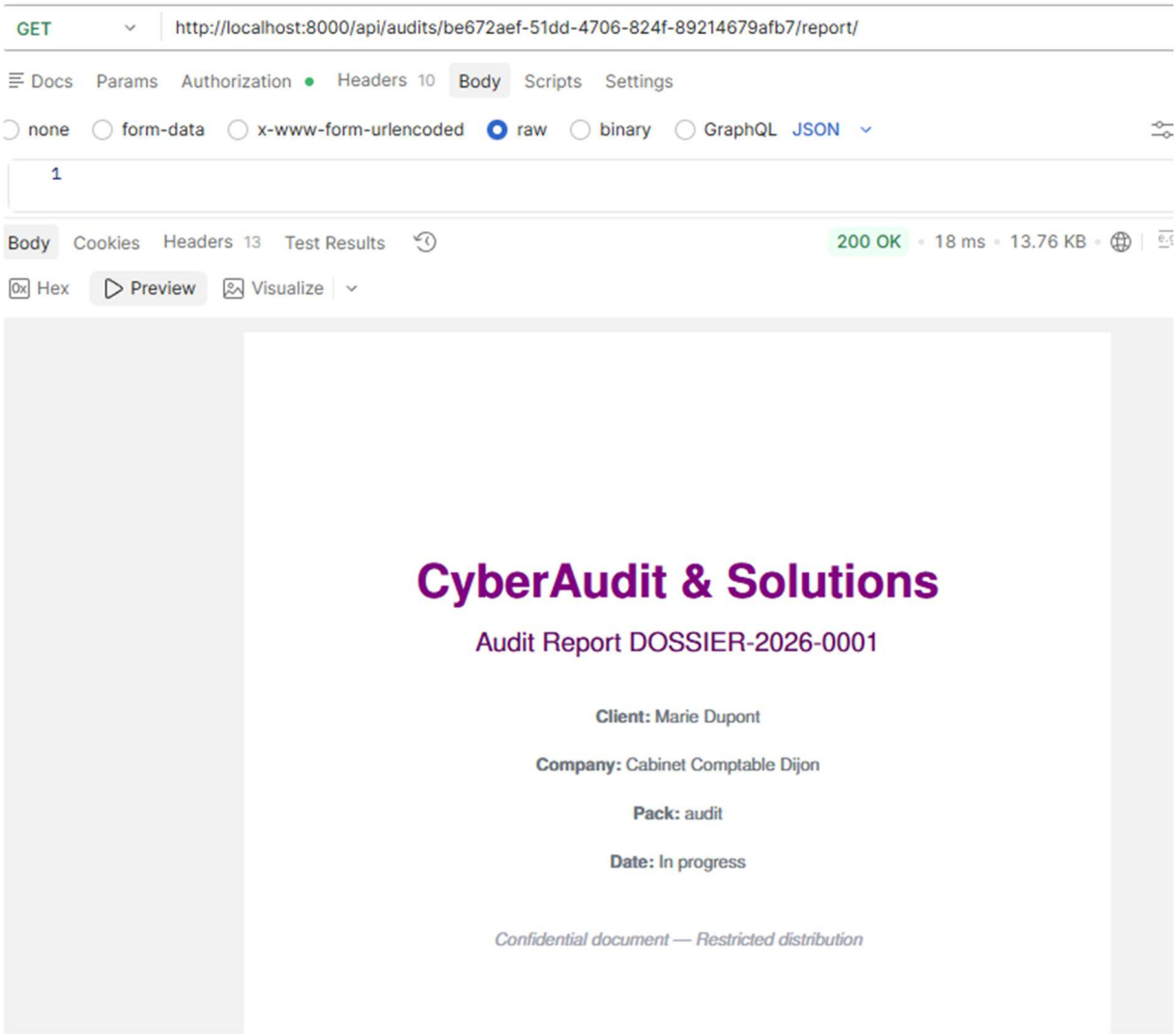
20) Generate the report as a PDF

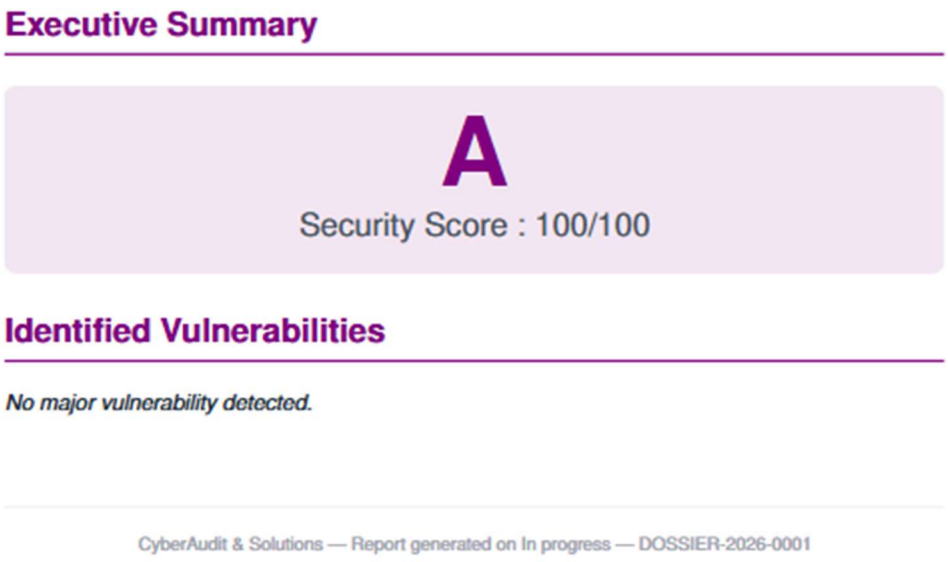


Status **202** (not 200 or 201) means that the task is **accepted and in progress** ; PDF generation is asynchronous via Celery .

- A report is created with a UUID: 7bb50516-8f7e-4dbb-aa8d-cecdf4ef5ed6.
- " status ": " generating ", the PDF is being created in the background.
- " security_score ": 100 and "grade": "A", automatically calculated score.
- " findings_count ": 0, no vulnerabilities detected (normal, it's a test).

21) Download the PDF report



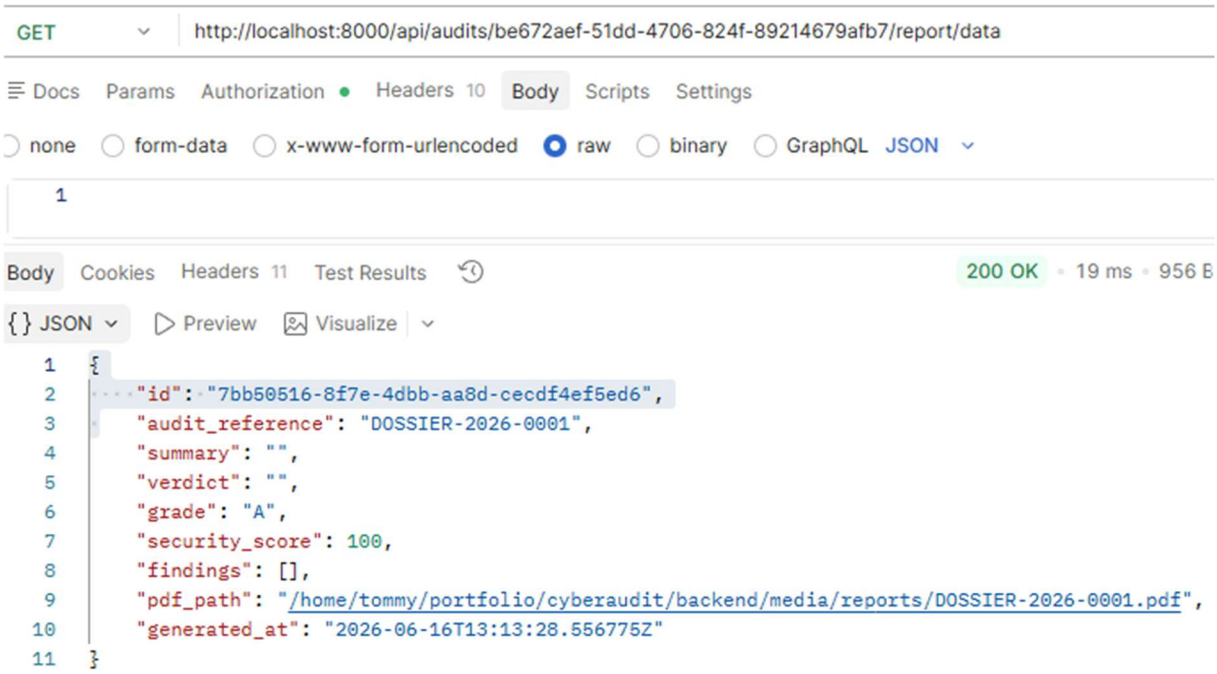


What this test shows:

- The PDF is returned directly in the response **(13.76 KB)**; Postman displays it in Preview.
- The cover page contains all the data for the file:
    - **Client:** Marie Dupont.
    - **Company:** Accounting Firm Dijon.
    - **Package:** audit.
    - **Reference:** DOSSIER-2026-0001.
    - Notice *"Confidential document. Restricted distribution".*
- The **Executive Summary** poster:
    - Grade **A** , Security Score **100/100.**
    - *"No major vulnerability detected ."*

This test validates the complete chain: request creation → status change → asynchronous PDF generation via Celery → report download.

22) Report data



200 OK : Report data retrieved!

What this test shows:

- Returns the **JSON metadata** of the report (unlike /report/ which returns the binary PDF).
- audit _reference: DOSSIER-2026-0001, link to the file.
- grade: "A" and security_ score: 100, consistent with the generated PDF.
- findings: [] no vulnerabilities recorded.
- pdf _path: exact path of the file on the server → /media/reports/DOSSIER-2026-0001.pdf.
- generated _at: 2026-06-16T13 : 13: 28, generation timestamp.
- The summary and verdict fields are empty; these fields need to be filled in by the admin to enrich the report.

Difference between the two Endpoint reports:

Endpoint	Return
/report/	The binary PDF file (download)
/report/data/	JSON metadata (for frontend display)

All endpoints have now been successfully tested! The CyberAudit & Solutions API is fully functional from end to end.